UNIVERSIDAD TECNOLOGICA NACIONAL

Distance Learning Program in Computer Programming

Capstone Project

Dog Walk App

Group dog walks and bookings for pet owners in Buenos Aires

Team

Gastón Lell

Gabriel Lovera

Juan Cruz Leal

November 2026

Contents

Section	Page
Executive Summary	3
1. Scope and Requirements	4
2. Architecture and Modules	5
3. Data Model	6
Table Relationships	7
4. Business Rules and Security	8
5. Test Plan and Results	9
6. Coverage and Publishing	10
7. Pets and Walk Search	11
8. Booking and Cancellation	12
9. Completing and Reviewing a Walk	13
10. Setup and Demo Walkthrough	14
11. Team, Scope and Conclusions	15
Appendix: API and References	16
Appendix: Additional Screenshots	17

Executive Summary

Dog Walk App, also called Paseadores, is a web app for organizing group dog walks in Buenos Aires. Walkers publish the neighborhood, time, meeting point, price and capacity. Owners add their pets and book one spot at a time. Each pet takes one spot.

Walkers set their service areas, publish walks and check their schedule. Owners can add a pet when they sign up, search by neighborhood, date or time, and book for one of their pets. My bookings separates upcoming, in-progress and past walks. An owner who took part can review a completed walk from History.

The Problem We Wanted to Solve

When a walk is arranged through messages, the schedule, meeting point and last-minute changes can end up in different conversations. It is also hard to tell how many pets have booked. We put those details on the walk page and gave both roles a place to check what is coming up.

Project Goal

Build a web app backed by a relational database where walkers can organize group walks by neighborhood and owners can book only when there is room.

Specific Goals

• Let walkers publish in the neighborhoods they cover.
• Let owners search by neighborhood and book for their own pets.
• Keep a history and check permissions for cancellations, completion and reviews.
• Use two concurrent requests to verify that only one can book the last spot.

We tested both roles with sample accounts and a local database. The screenshots and results are included below. We will add links to the public app and demo video when they are available.

1. Scope and Requirements

The app has two roles: pet owner and walker. Each has its own navigation. The neighborhood catalog is public, but account data, walks and bookings require sign-in.

Requirement	Expected Behavior
RF01. Accounts and profiles	Sign up as an owner or walker, sign in and edit your own details. Other users cannot see the walker's email or phone number.
RF02. Service coverage	Save neighborhoods and suggested values for price and capacity. Publishing uses those service neighborhoods.
RF03. Publishing	Create a future walk with duration, price, capacity and a meeting point. Confirm publication and show it in the schedule. Reject overlapping walks.
RF04. Pets and search	Add and edit your own pets. Search for future walks by neighborhood, date or time and view the details.
RF05. Booking	Book for one of your pets if a spot remains and the start time has not arrived. See Upcoming, In progress and History.
RF06. Cancellation and completion	Free the spot when a booking is cancelled in time. The walker can start with booked pets and complete a walk in progress.
RF07. Reviews	Accept one review per participating owner and completed walk. Show the review and update the walker's rating.

Quality Criteria

The server checks permissions and availability, and database constraints prevent duplicates. We also tested the layout on a narrow screen. The setup guide explains how to repeat the walkthrough with a fresh database.

Scope Limits

Prices are shown in Argentine pesos. The map shows approximate neighborhood locations and the meeting point is given as text. The app does not include online payments, live location tracking, chat or a native mobile app.

2. Architecture and Modules

The interface uses React, TypeScript and Vite. Spring Boot handles the API rules, and PostgreSQL stores the data. Flyway runs the migrations. In development, Vite proxies /api to the backend. The production Dockerfile builds the frontend and serves it with Spring Boot.

Layer	Responsibility
Interface	Screens for sign-in, profiles, pets, search, walk details, schedules, bookings and reviews. Shows results and validation messages.
Controllers	Read HTTP requests, identify the signed-in user and convert service results or errors to API responses.
Services	Check rules such as coverage, schedule overlaps, pet ownership and capacity before allowing an operation.
Persistence	Repositories read and write PostgreSQL records. Constraints and transactions enforce rules that the interface cannot guarantee.

Repository Layout

Directory	Contents
frontend/	Screens, routes, forms and HTTP client.
backend/	Controllers, services, entities, repositories, migrations and Java tests.
database/	DDL, seed data, verification queries and editable DBML schema.
docs/	Proposal, design decisions, setup guide, API contract and test evidence.

Implementation Decision

The walks module covers publishing, search and scheduling. BookingService coordinates bookings and cancellations. WalkPetService checks the rules for each booking. After publishing a walk, the walker sees a confirmation and a shareable link. Their schedule separates upcoming walks from history.

3. Data Model

We use a relational database. The tables below show the main relationships. The Flyway migrations define the exact keys and types, and the repository includes an editable DBML model.

Table	Relationships and Main Fields
zones	Neighborhood catalog. A neighborhood may be linked to users, walks and service areas.
users	Owner (CLIENT) or walker (WALKER), home neighborhood and profile details. Walkers may have default prices, capacities and rating data.
walker_zones	Links a walker to each neighborhood where they offer walks.
pets	Each pet has one owner, a name and optional size, breed and notes.
walks	Each walk belongs to one walker and one neighborhood. Stores time, duration, meeting point, price, capacity and status.
walk_pets	Links a pet to a walk. Booking status keeps the cancellation history and lets us reactivate a booking.
reviews	A review belongs to a walk and an owner. It stores a rating, comment and date.

Data Integrity

Email addresses are unique regardless of case. The walker and neighborhood, walk and pet, and walk and review author pairs are also unique. Foreign keys require referenced records to exist. Checks require positive capacities and durations, non-negative prices and ratings from one to five.

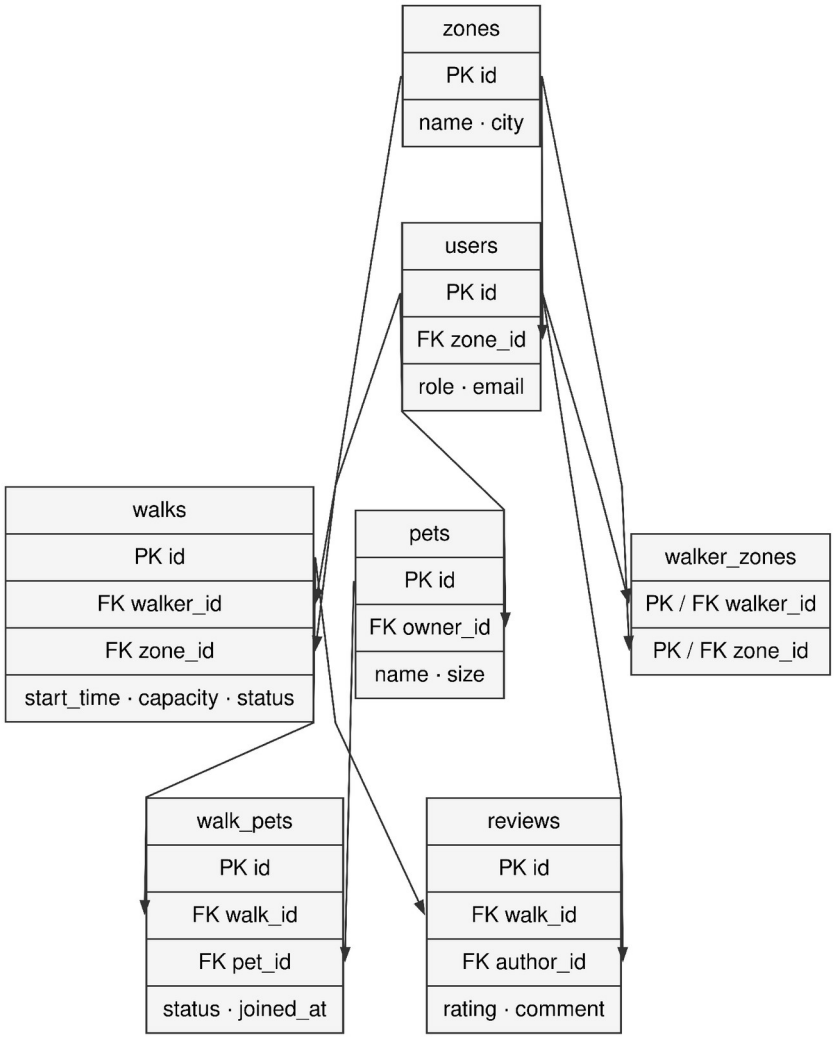
Initial Data

The first migration creates the schema and the second loads the neighborhoods. We started the demo with an empty database and added the accounts, pets and walks in the app. Prices are stored as decimals and schedules use local Buenos Aires time.

Design reference: the repository data model and diagram.

Table Relationships

Each arrow runs from a primary key to the field that references it. The diagram shows the keys and common fields. The first migration contains the full definitions.



Simplified diagram based on the SQL schema. PK: primary key. FK: foreign key.

How We Use These Relationships

Every user has a home neighborhood. A walker can also cover several service neighborhoods. Each walk belongs to the walker who published it and to the neighborhood where it takes place.

A pet can join several walks, and a walk can have several pets. The walk_pets table records those bookings and their status. An owner can write one review per walk, even if they booked more than one pet.

4. Business Rules and Security

Publishing and Scheduling

A walker can publish in a neighborhood they cover, at least two hours before the start. A walk must last at least 30 minutes, in 15-minute increments. The service takes a lock for that walker before checking for overlapping walks.

Booking and Cancellation

Each booking is for one of the owner’s pets. The backend locks the walk row, then checks that the walk is still published, that its start time has not arrived and that there is room. If two requests compete for the last spot, the second waits and checks availability again.

An owner can cancel a scheduled booking when more than two hours remain. The cancellation stays in the history and frees a spot. Rebooking for the same pet reactivates the record. Cancelling a walk also cancels its active bookings.

Walk States and Reviews

State	When It Applies
Published or full	A scheduled walk. The state changes automatically when the last spot is taken or a spot opens up.
In progress	The walker can start a published or full walk if at least one pet is booked.
Completed	The walker can finish a walk that is in progress. An owner who took part can leave a review.
Cancelled	The walker can cancel a published, full or in-progress walk. Active bookings are cancelled and the history is kept.

Only an owner who took part can review a completed walk, and they can review it once. Saving the review updates the walker’s average rating and review count.

Access Control

The API identifies signed-in users with JWTs and hashes passwords with BCrypt. Walk search, walk details and other walkers’ profiles require sign-in. Walker profiles omit private contact details. The server checks role and ownership for write operations. Database credentials and the JWT key come from the environment.

5. Test Plan and Results

We tested with Java 21, PostgreSQL 17 and Node.js 24. We used one walker account and one owner account and checked the owner screens at 390 px.

Check	Result
Backend	211 tests passed with Java 21 and PostgreSQL 17, including integration tests.
Frontend	51 tests passed. The production build and lint also passed.
Overlapping walks	Two concurrent attempts to publish overlapping walks could not both succeed.
Last spot	Only one of two concurrent requests could book the last spot.
Permissions and states	We checked cancellation, rebooking, state changes and rejection at the scheduled start time.
Browser and mobile	We tried both roles and checked search, bookings and reviews at 390 px.

Screenshots and Test Accounts

Using sample accounts, we published a walk, booked the last spot, cancelled and booked again. We watched the booking move from Upcoming to In progress and then to History, where we left a review.

Booking Behavior

Once the start time arrives, the walk no longer appears in search and new bookings are rejected. In My bookings, In progress follows the walker’s status update. Completed walks and cancelled bookings appear in History.

6. Coverage and Publishing

We set Palermo as the walker's service neighborhood, with one spot and a reference price. We published a walk and checked the schedule. When we tried to publish another walk at an overlapping time, the API rejected it.

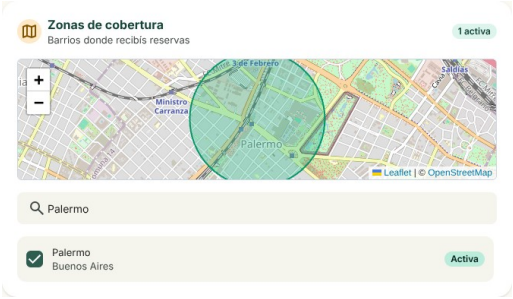


Figure 1. Palermo in the walker's service area.



Figure 2. Confirmation after publishing the walk.

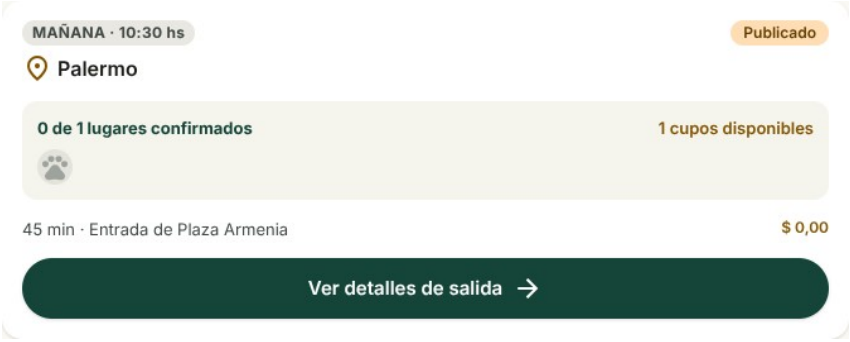


Figure 3. The published walk in the walker's schedule, with time, capacity and meeting point.

7. Pets and Walk Search

We registered Sofía and added Luna in the second step. We searched by neighborhood and date. The result showed the time, meeting point and available spot.



Figure 4. Luna appears in My pets.



Figure 5. A result in Palermo with its time, price and one available spot.

8. Booking and Cancellation

We booked the only spot for Luna. We cancelled from Upcoming, found the cancellation in History and checked that the spot was available again. Then we rebooked for the same pet.



Figure 6. The walk details show that the only spot is taken.



Figure 7. The confirmed booking appears under Upcoming in the owner's account.



Figure 8. The cancelled booking remains in History.

9. Completing and Reviewing a Walk

After we rebooked, the walker started and completed the walk. In the owner’s history, we selected Review the walk, submitted five stars and checked the rating on the walker’s profile.

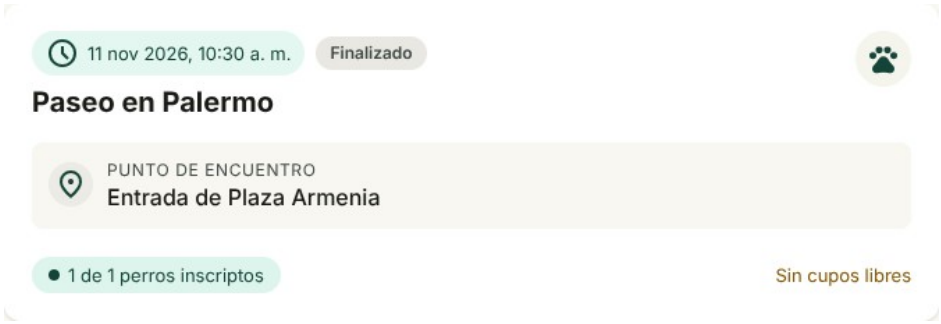


Figure 9. The walker’s screen shows the walk as completed.



Figure 10. The profile shows a five-star average and one review.



Figure 11. The published review on a 390 px screen.

The accounts and pets are fictional. We used the meeting point as an example.

10. Setup and Demo Walkthrough

Database and Configuration

Create a database in PostgreSQL 14 or later. The backend needs Java 21 and database connection settings. Flyway creates the schema and loads the neighborhoods on startup.

```
DB_URL=jdbc:postgresql://localhost:5432/paseadores
DB_USER, DB_PASSWORD and JWT_SECRET: use your local values.
```

Set these variables in the shell or in an untracked backend/.env file. The Dockerfile packages the frontend and backend together. PostgreSQL runs separately. The repository guide covers both ways to run the app.

Backend and Frontend Terminals

From the repository root, open two terminals. In the first, start the backend with the Maven wrapper. In the second, install the frontend dependencies using Node.js 24 and pnpm 11.25.0, then start Vite.

```
cd backend && bash mvnw spring-boot:run
```

```
cd frontend && pnpm install --frozen-lockfile && pnpm dev
```

On Windows, you can use mvnw.cmd. Open http://localhost:5173. Vite proxies /api to Spring Boot at http://localhost:8080. Swagger is available at /swagger-ui/index.html on the backend.

Demo Steps

1. Register as a walker, add Palermo to your service area and publish a walk.
2. Register as an owner, add a pet, search by neighborhood and date, and book.
3. Cancel from Upcoming while more than two hours remain, then book again.
4. Start the walk and check In progress. Complete it and select Review the walk from History.

11. Team, Scope and Conclusions

How We Worked

We planned the work in ClickUp and collaborated on GitHub. Juan Cruz worked on the analysis and how the modules fit together. Gabriel worked on the schema and database scripts, and Gastón organized the project structure. We all contributed to features, testing and fixes.

We worked on the owner flow, bookings and the walker's schedule. We used the Stitch design for the owner's search, details and booking screens. Walkers use their own profile and schedule.

Before Submission

Pending	What We Still Need to Check
Team review	Review the report together and agree on the walkthrough for the submission.
Public deployment	Deploy the app with persistent PostgreSQL and verify sign-in and booking from the public URL.
Demo video	Record both roles using sample data. Check the link and add it to the README and report.

Conclusions

Showing open spots on screen was only part of the problem. The important check was at confirmation: if two people requested the same last spot, the database had to accept only one. We tested that with two separate requests.

We checked that capacity and status changes are visible in both accounts and that owners can leave a review from History. Before submission, we still need to test the public deployment and add the video link.

Appendix: API and References

The API uses JSON and a Bearer token for protected operations. Swagger is available at /swagger-ui/index.html. The table shows the main routes for the demo.

Method and Route	Operation
POST /api/auth/register POST /api/auth/login	Create an account or sign in.
GET /api/zones GET / PUT /api/walkers/me	Read the catalog or manage your own service coverage.
GET / POST /api/walks GET /api/walks/mine	Search or publish walks and read the walker’s schedule.
POST /api/walks/{id}/join/{petId} GET /api/walks/bookings/mine	Book for one of your pets. List Upcoming (UPCOMING), In progress (IN_PROGRESS) or History (HISTORY).
DELETE /api/walks/{id}/join/{petId}	Cancel your own booking.
PATCH /api/walks/{id}/status POST /api/walks/{id}/reviews	Change a walk’s state or review a completed walk as a participating owner.

References

Team repository: https://github.com/loveragabriel/trabajo-practico-integrador

Proposal, architecture and scope

Walk and booking controller

Local setup guide

Universidad Tecnológica Nacional visual identity guide

OpenStreetMap data attribution

Appendix: Additional Screenshots

Walkers can see the current state and the pets booked on a walk. When the walker starts it, the owner sees the booking under In progress.



Figure 12. The walk is in progress in the walker's view.

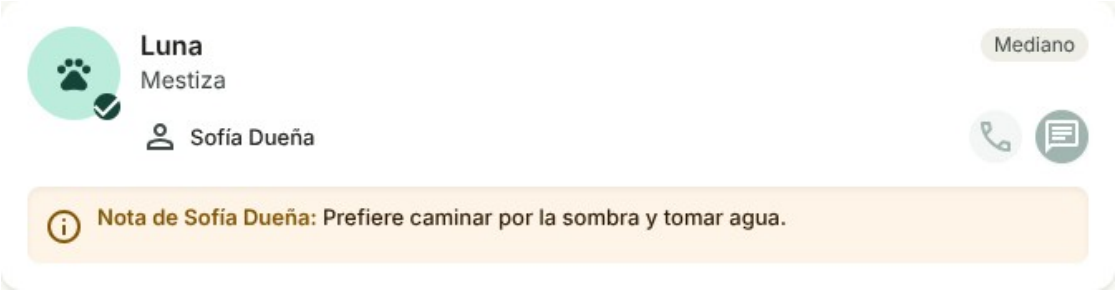


Figure 13. A booked pet and the notes shown to the walker.



Figure 14. The owner sees the booking under In progress.